CPU 동작

이 내용은 특별히 외울 것은 없고 8086 CPU를 이용하여
어셈블리 언어와 어셈블러를 통한 기계어로의 변환 방법, 기계어의 구조 및
CPU가 내부 레지스터를 이용해 어떻게 기계어를 실행하는지만 이해하면 됩니다.

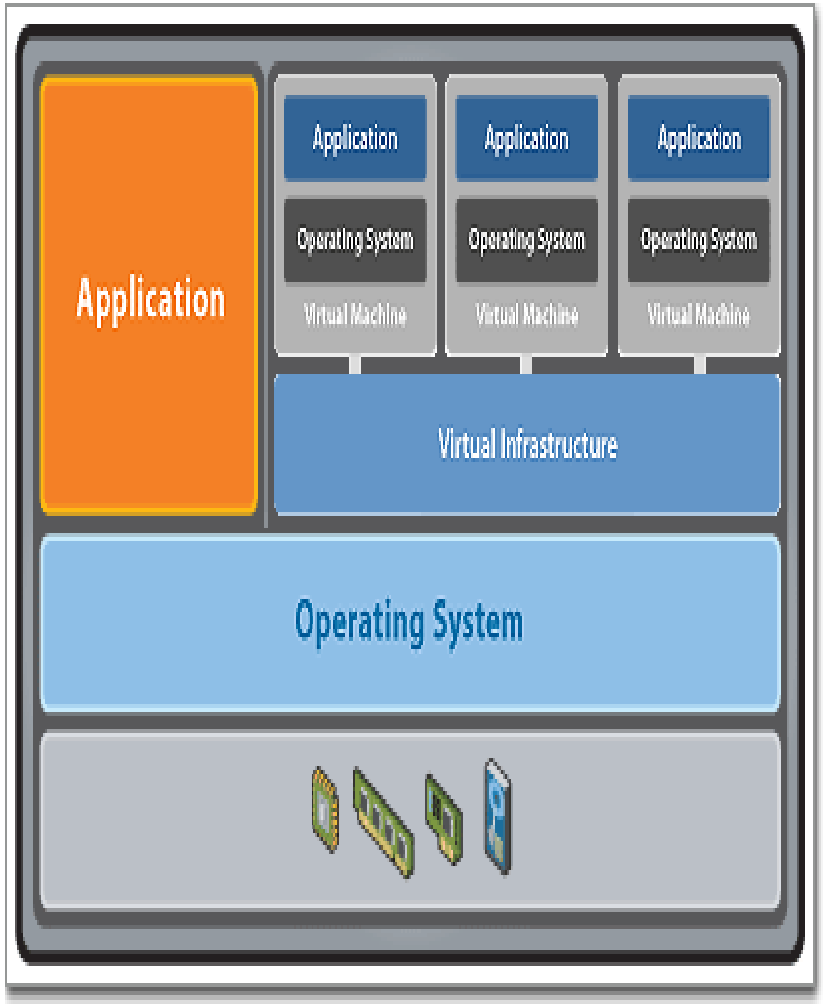
Presented by
오 세 춘

어셈블리 언어 프로그래밍(8086)

1. 실습 개요

■ 가상 MS-DOS 환경 구축의 필요성

- ✓ 8086 계열 이후의 프로세서는 멀티 태스크 환경을 지원 → Windows
  - ➢ 멀티 태스크를 위해 자원의 직접적인 사용을 제한
  - ➢ 8086에서는 MS-DOS를 사용하여 싱글 태스크의 작업만 가능 → DEBUG 프로그램 사용 가능
- ✓ 근래의 PC 환경에서는 어셈블리 언어 사용이 매우 제한적
  - ➢ 하드웨어의 급속한 발전
  - ➢ 고급 언어용 다양한 컴파일러의 등장
  - ➢ Visual Studio 등에서 어셈블리 언어와 관련된 내용 지원
    - ❖ 우리가 간단히 실습하기에는 패키지의 규모가 너무 크고 설치 시간도 너무 많이 소요
- ✓ 따라서 DOSBox 라는 8086 CPU 시뮬레이터(가상 머신)를 설치하여 실습
- ✓ VMware, Virtual Box 등의 훌륭한 가상머신 소프트웨어가 있으나 이들은 규모가 너무 커서 우리는 MS-DOS 전용 가상머신 S/W를 사용



1. 실습 개요

■ 실습 절차 및 목적

1) MS-DOS의 가상머신(Virtual Machine)인 DOSbox를 설치하여 MS-DOS 시절의 환경을 구축

2) 사용되는 프로그램들

- MASM.EXE (매크로 어셈블러 프로그램)
  - *.asm (어셈블리어 소스코드)을 목적파일(object)로 변경하는 어셈블러 프로그램
- LINK.EXE (링커 프로그램)
  - *.obj (목적파일)을 하나의 실행파일(*.exe)로 변경하는 프로그램
- DEBUG.EXE (디버깅 프로그램)
  - 실행파일을 한 명령어 씩 또는 Breakpoint까지 수행하면서 디버깅 할 수 있는 프로그램

3) 몇 개의 어셈블리 예제 소스 프로그램을 실행파일로 생성

4) DEBUG 프로그램을 이용하여 예제 프로그램의 명령어들을 하나씩 순서대로 실행 시키면서 8086 CPU 내부의 각종 레지스터들의 움직임을 파악

5) 이를 통해 CPU의 동작 원리에 대한 이해를 하는 것이 본 실습의 목적

2. 실습 환경의 구축

■ DOSBox 다운로드 및 설치 – 1/2

여기를 누르면 DOSBox 프로그램을 다운로드 받을 수 있음. 윈도우, Mac, 다양한 리눅스용 버전이 있음

1) e-class 강의자료 "8086 실습"에서 첨부자료를 다운로드

2) 다운로드 받은 내용을 압축을 풀어서 D:₩8086 폴더에 복사

   ➢ 또는 본인이 원하는 드라이브도 가능

3) DOSBox의 설치

   ① D:₩8086 폴더의 "DOSBox0.74-win32-installer" 실행

      ❖ MS-DOS 가상머신 프로그램의 설치

   ② 바탕화면의 DOSBox 아이콘 클릭하여 실행

   ③ 생성된 DOS창에 "**mount c d:₩8086**"을 입력

      ❖ D 드라이브의 8086 폴더를 가상 머신 소프트웨어인 DOSBox의 C 드라이브로 설정하는 내용임

   ④ DOS 창에 "**C:**"를 입력하고 "**dir**" 명령을 입력해 파일들이 존재하는지 확인

D:₩ 이외의 다른 경로에 설치도 가능

입력 = 명령 + 엔터키

MS-DOS에서는 대소문자를 구별하지 않는다

2. 실습 환경의 구축

■ DOSBox 다운로드 및 설치 – 2/2

① D:₩8086 폴더의 "DOSBox0.74-win32-installer" 실행

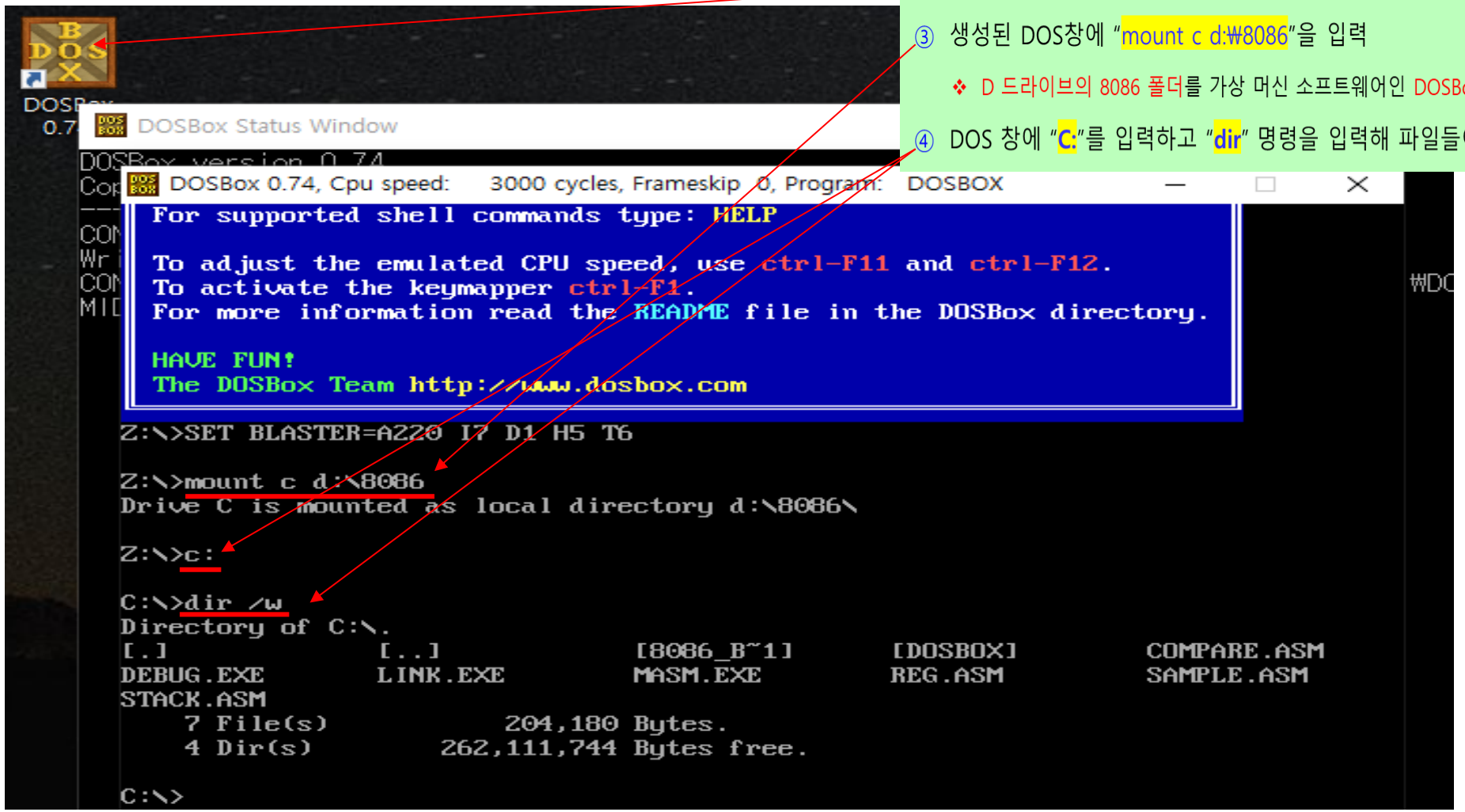
❖ MS-DOS 가상머신 프로그램의 설치

② 바탕화면의 DOSBox 아이콘 클릭하여 실행

③ 생성된 DOS창에 "mount c d:₩8086"을 입력

❖ D 드라이브의 8086 폴더를 가상 머신 소프트웨어인 DOSBox의 C 드라이브로 설정하는 내용임

④ DOS 창에 "C:"를 입력하고 "dir" 명령을 입력해 파일들이 존재하는지 확인



2. 실습 환경의 구축

■ 주요 프로그램/파일의 설명

1) MASM.EXE

2) LINK.EXE

3) DEBUG.EXE

4) REG.ASM

   ➢ 몇가지 명령어를 실습해 보는 어셈블리 소스

5) STACK.ASM

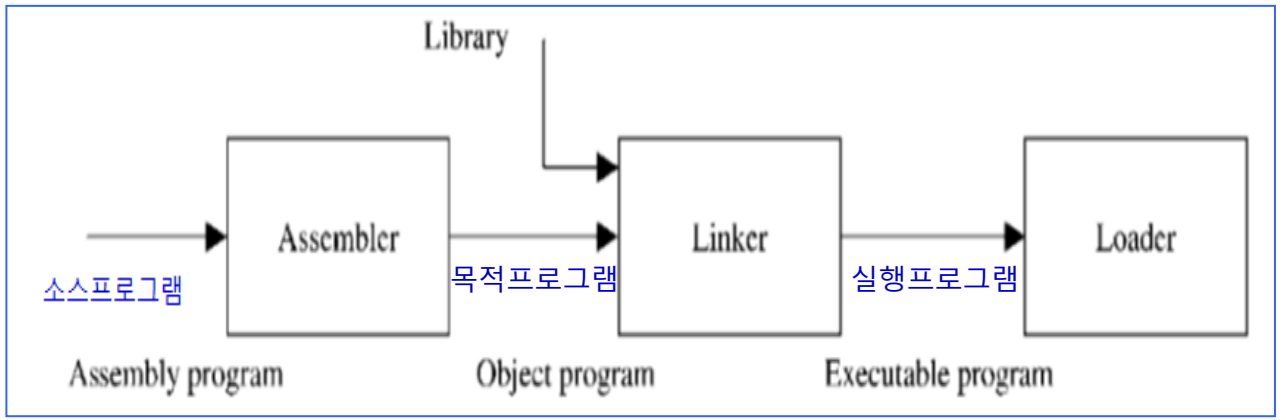
   ➢ 스택 포인터의 동작을 실습해 보는 어셈블리 소스

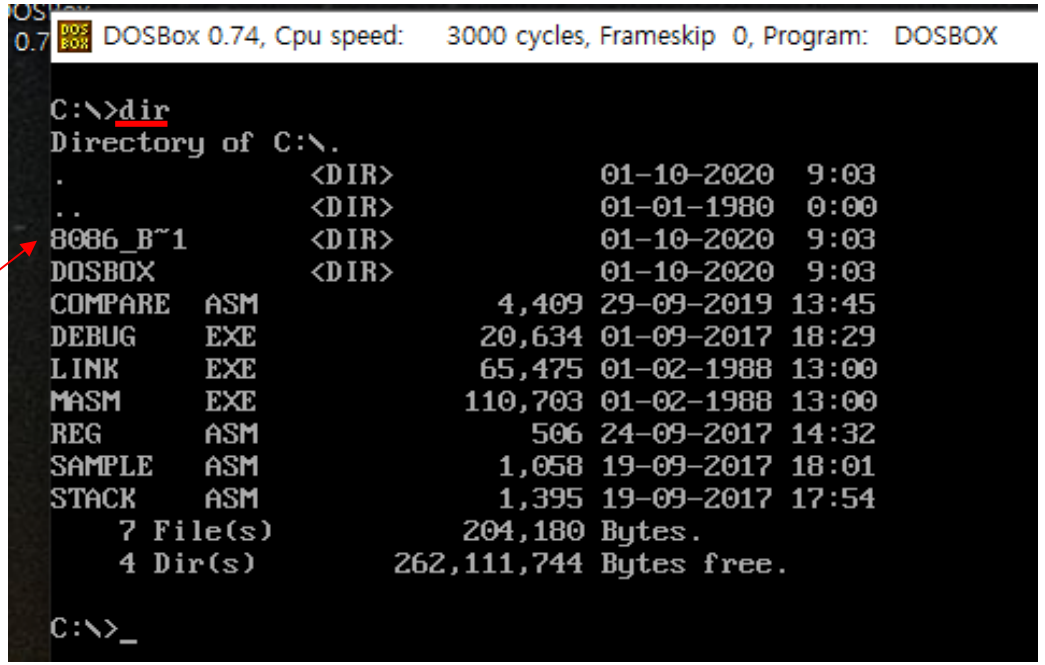
6) COMPARE.ASM

   ➢ 산술 연산 어셈블리 소스의 8086 버전

7) SAMPLE.ASM

   ➢ 이름 출력 프로그램 어셈블리 소스





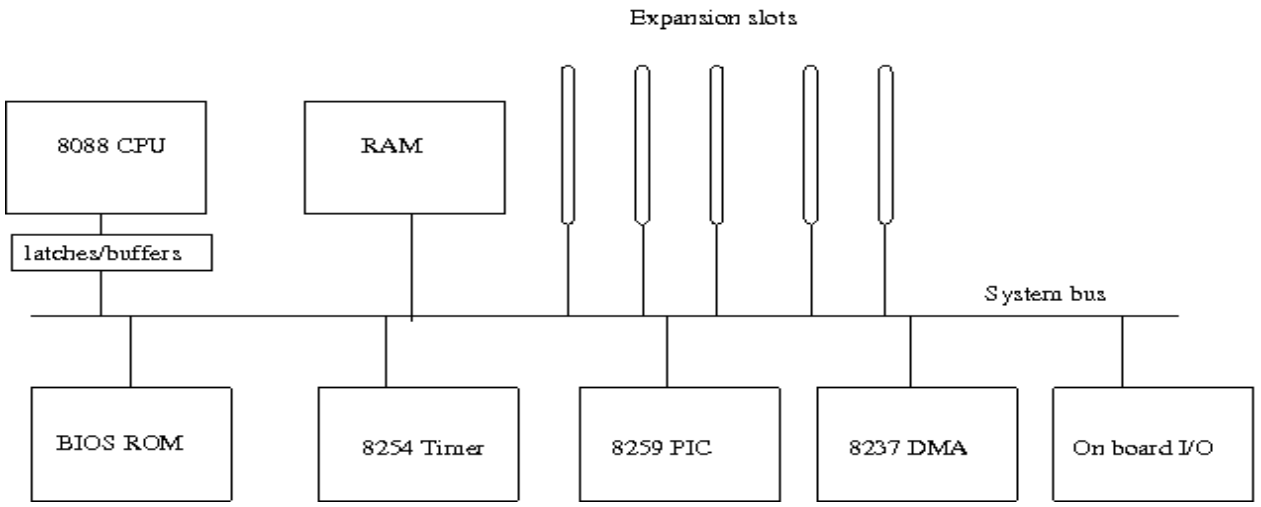
8086_bios_and_dos_interrupts 설명자료 디렉토리
- 8086_bios_and_dos_interrupts
- 8086_instruction_set

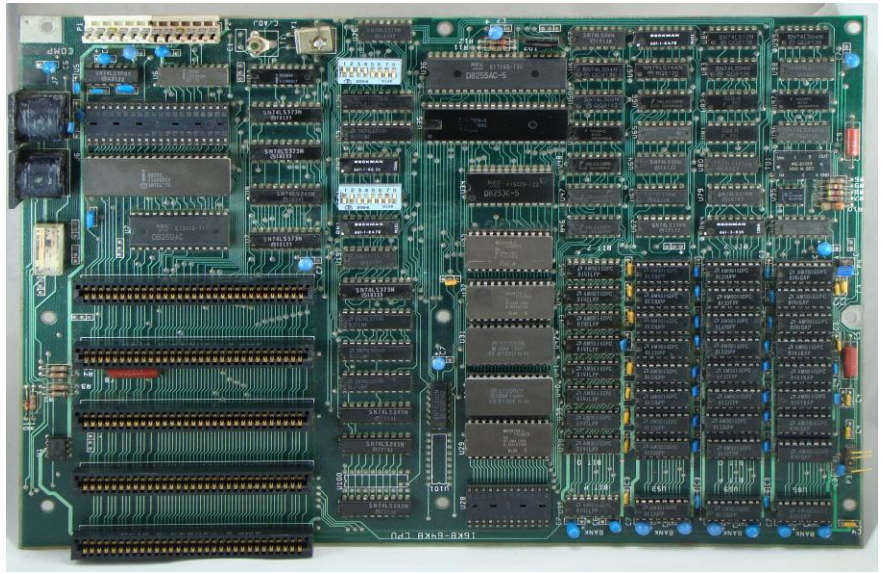
3. 8086 CPU

- i8088 CPU를 이용한 IBM Personal Computer





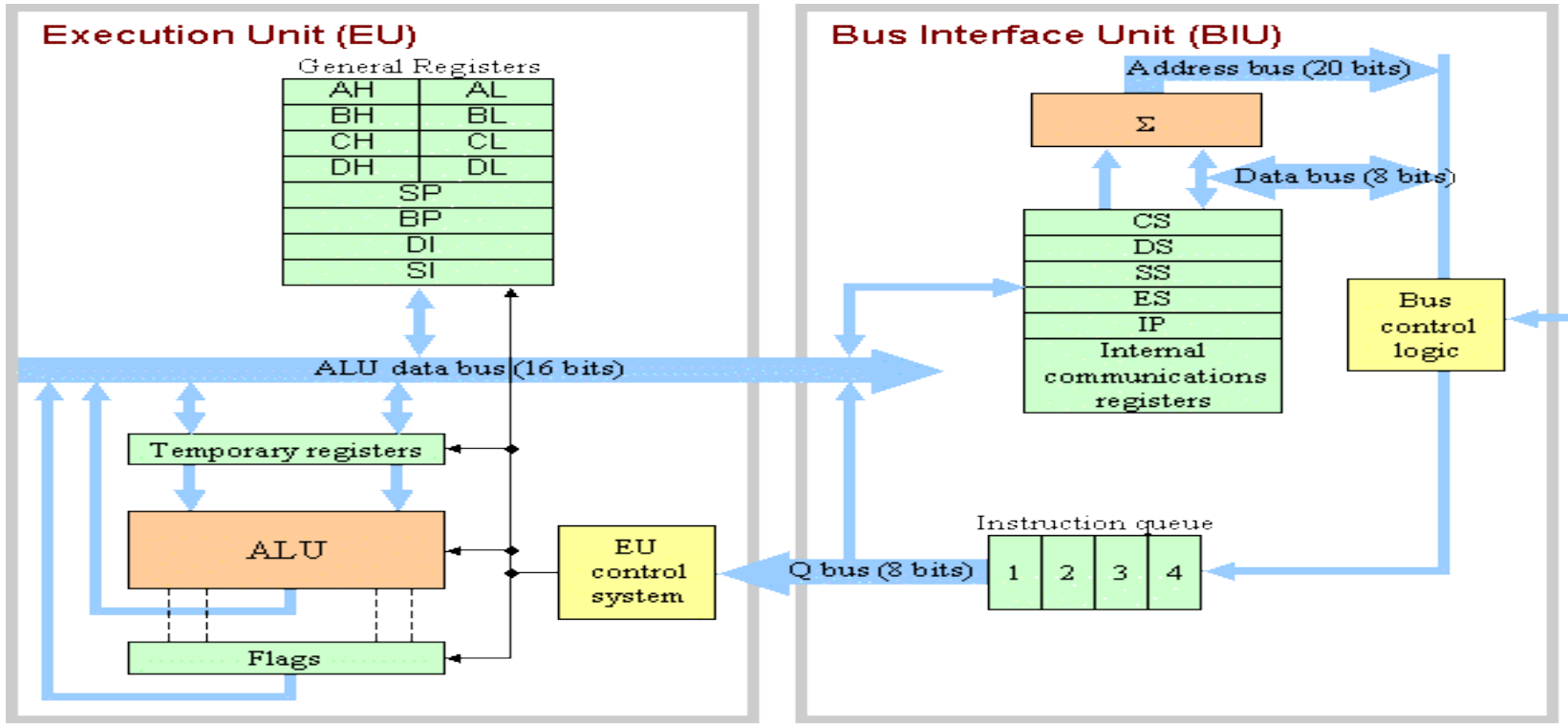


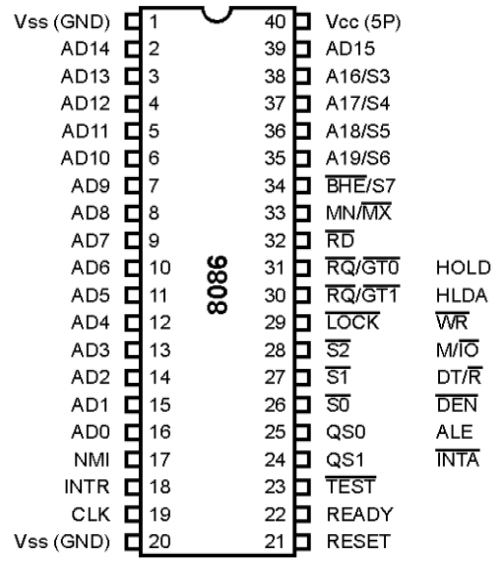


■ 인텔 8088/8086 프로세서 내부 구조

MAR = 20비트
MDR = 8비트(i8088), 16비트(i8086)

✓ i8088은 8비트 Only 외부 데이터 버스





-- Static registers (groups of D Flip-Flops) used to hold or transfer binary data

-- Logic gate circuits designed to perform arithmetic or logical functions

-- Logic gate circuits designed to provide internal control to processor

-- Internal data busses used to pass information between components

Reserved word (예약어)

■ 인텔 8088/8086 프로세서의 레지스터 – 1/4

1) 주소 지정 레지스터

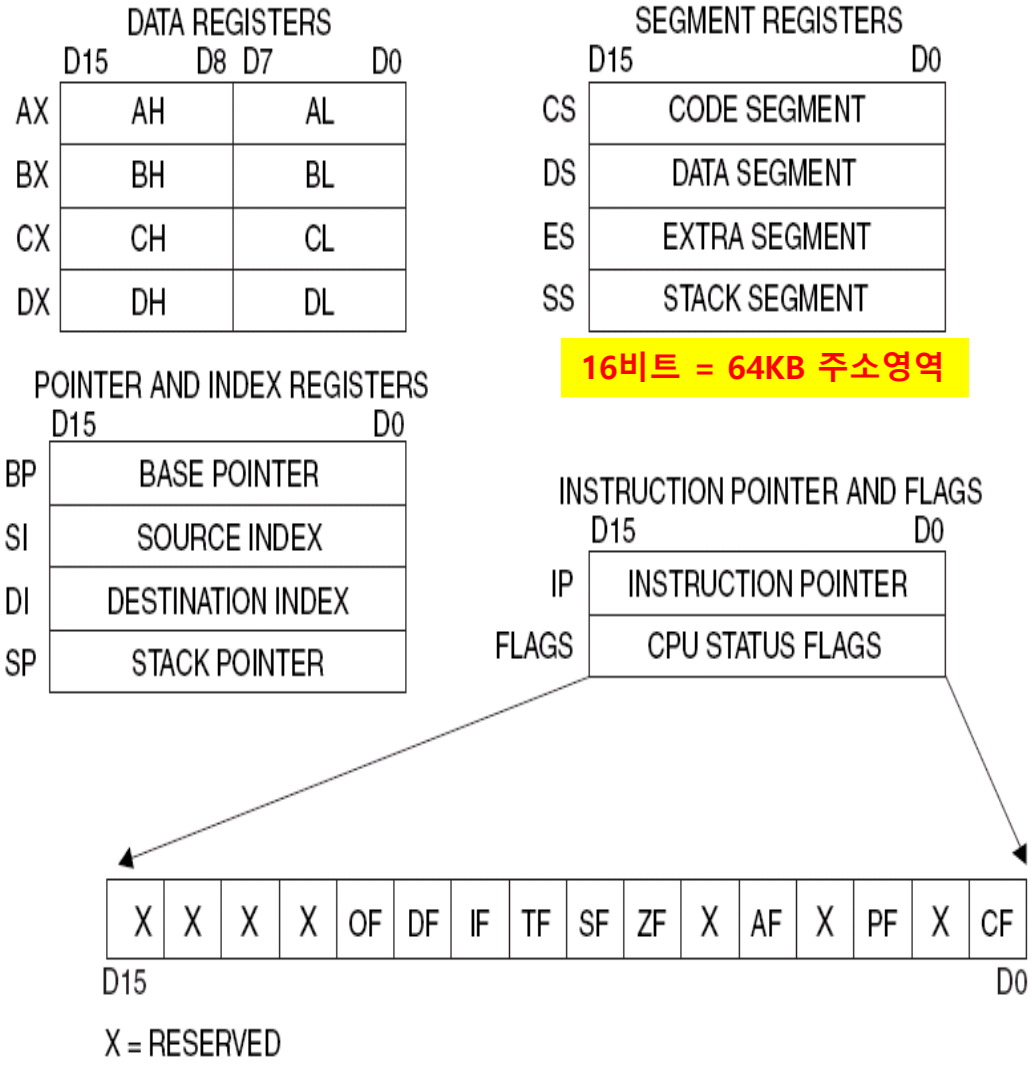
- 프로그램의 실행 → CS:IP
- 기존 데이터의 처리 → DS:SI
- 추가 데이터의 처리 → ES:DI
- 스택의 처리 → SS:SP
- 추가적인 주소 지정 → BP

2) 범용 레지스터

- AX (AH, AL)
- BX (BH, BL)
- CX (CH, CL)
- DX (DH, DL)

3) Status Register (Flags)

- 상태 레지스터

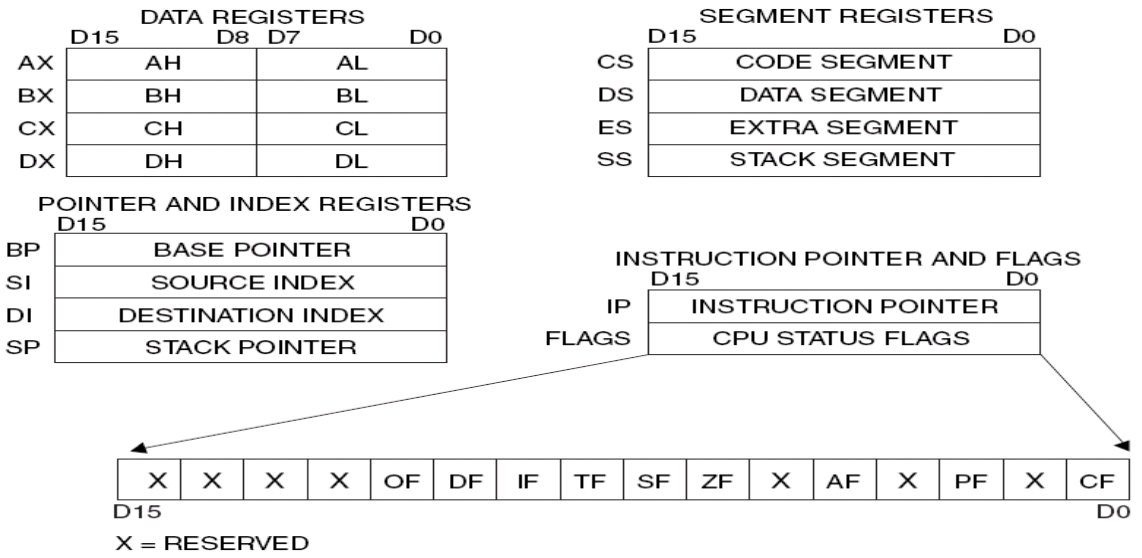


■ 인텔 8088/8086 프로세서의 레지스터 – 2/4

1) 주소 지정 레지스터

- 8086은 총 20개의 시스템 주소 버스선을 사용(A_{19}~A_0)
- 세그먼트 레지스터(16bit)와 포인터 레지스터(16bit)를 이용해 20비트의 주소 생성
  - 2^{20}=1MB
  - 세그먼트 레지스터로 시작 위치를 결정하고 포인터 레지스터로 오프셋을 결정
  - 세그먼트 레지스터 XXXXh + 포인터 레지스터 YYYYh → XXXX0h + YYYYh
- 세그먼트, 포인터 레지스터들은 항상 짝으로 실행
  - 프로그램의 실행 → CS:IP
  - 기존 데이터의 처리 → DS:SI
  - 추가 데이터의 처리 → ES:DI
  - 스택의 처리 → SS:SP
  - 추가적인 주소 지정 → BP

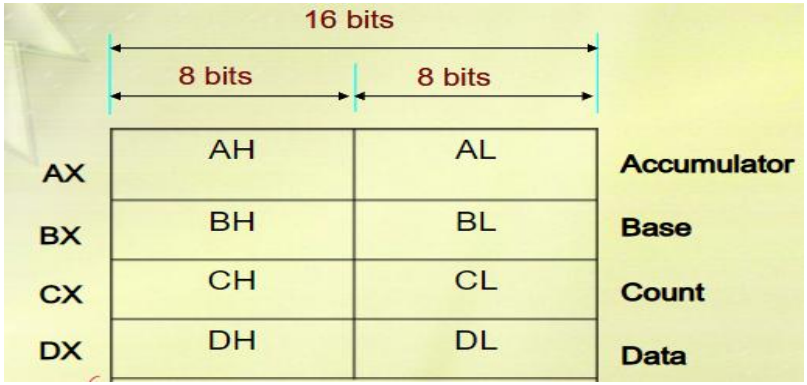
h = hexadecimal = 16진수



■ 인텔 8088/8086 프로세서의 레지스터 – 3/4

2) 범용 레지스터

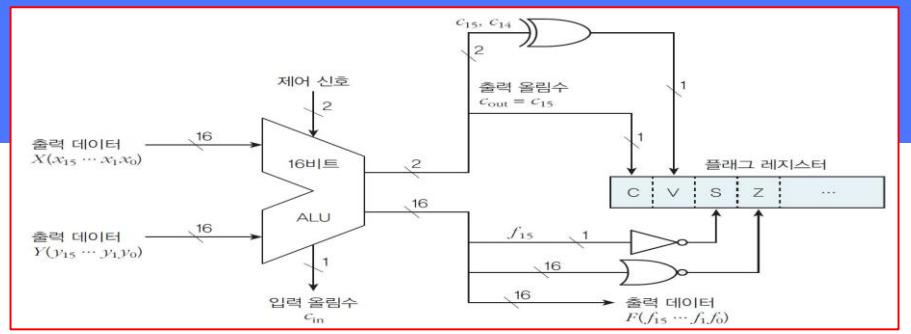
- 16비트 또는 개별 8비트 레지스터로 사용 가능
- 다음의 경우를 제외하고 일반 목적으로 사용



Register	Purpose
AX	Word multiply, word divide, word I /O
AL	Byte multiply, byte divide, byte I/O, decimal arithmetic
AH	Byte multiply, byte divide
BX	Store address information
CX	String operation, loops
CL	Variable shift and rotate
DX	Word multiply, word divide, indirect I/O (Used to hold I/O address during I/O instructions. If the result is more than 16-bits, the lower order 16-bits are stored in accumulator and higher order 16-bits are stored in DX register)

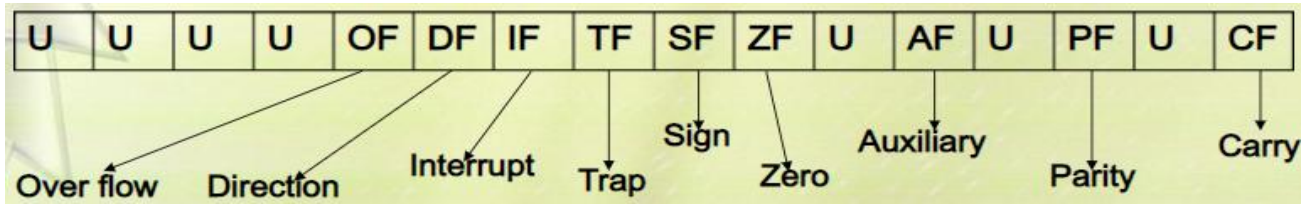
■ 인텔 8088/8086 프로세서의 레지스터 – 4/4

3) 상태 레지스터 (Status Register 또는 Flag Register)



- **OF** (Overflow Flag)
  - 유부호 연산 결과 범위 초과 → 1
- **DF** (Direction Flag)
  - SI, DI의 자동 증가(0)/감소(1) 방향 지정
- **IF** (Interrupt Flag)
  - 외부 인터럽트의 허용(1)/mask(0)
- **TF** (Trap Flag)
  - 1이면 하나의 명령만 실행하고 정지 → 디버깅
- **SF** (Sign Flag)
  - 연산결과가 양수(0)/음수(1)
- **ZF** (Zero Flag)
  - 연산결과가 0일 경우 1
- **AF** (Auxiliary Flag)
  - bit3에서 캐리 발생 → BCD 연산에 사용
- **PF** (Parity Flag)
  - 연산 결과의 패리티 체크 결과
  - 0 = Odd parity, 1 = Even Parity
- **CF** (Carry Flag)
  - MSB에서 캐리 발생시 1
  - 무부호 연산 결과 범위 초과

SV(Super Visor) Flag 없음 → Singe Tasking



4. reg.asm 프로그램

■ reg.asm 프로그램의 실행

화면 한쪽엔 탐색기로 ***d:₩8086***을 열고 한쪽에는 DOSbox 화면을 연다

- ✓ 윈도우에서 D:₩8086를 연다
- ✓ 윈도우의 메모장으로 reg.asm을 열어 내용을 파악
- ✓ 어셈블러 실행
  - 1) "masm reg" 를 입력
    - ① Object file name 입력에서는 그냥 엔터키
      - ☞ reg.obj 파일이 만들어 진다
    - ② LST 이름을 물으면 "reg"를 입력
      - ☞ reg.lst 파일이 만들어 진다
    - ③ CRF 이름을 물으면 그냥 엔터키
      - ☞ Cross Reference 파일의 생성은 생략
  - ① "link reg;" 을 입력
    - ❖ reg.exe 파일이 만들어 진다
- ✓ 윈도우에서 LST 파일의 내용을 확인(윈도우 메모장 이용)
  - ❖ 여기에서 소스와 기계어를 모두 확인 가능 → 연산코드, 오퍼랜드, 조건부 이동 등등....

```
DOSBox 0.74, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG

C:\>masm reg
Microsoft (R) Macro Assembler Version 5.10
Copyright (C) Microsoft Corp 1981, 1988.  All rights reserved.

Object filename [reg.OBJ]:
Source listing  [NUL.LST]: reg
Cross-reference [NUL.CRF]:

  48072 + 463283 Bytes symbol space free

       0 Warning Errors
       0 Severe  Errors

C:\>link reg;

Microsoft (R) Overlay Linker  Version 3.64
Copyright (C) Microsoft Corp 1983-1988.  All rights reserved.


C:\>debug reg.exe
-_
```

4. reg.asm 프로그램

① AL 레지스터에 숫자 2를 넣고
② AL = AL - 1
③ AL이 0이 될 때까지 ② 반복
④ AL이 0 이면 프로그램 종료

■ reg.lst 내용

주소 기계어

```
14 0000                 cseg segment            ; 코드 세그먼트의 선언
15 0000                 main proc far
16                          assume ss:sseg, ds:dseg, cs:cseg    ; 기타 세그먼트 레지스터들의 초기화
17
18 0000 B0 02               mov  al,2           ; 즉치(표현 방법이 다름) al 레지스터에 2 설정.  B0h = 10110000b
19 0002                 re_check:
20 0002 FE C8               dec  al             ; al 1 감소     FEh = 11111110b
21 0004 75 FC               jnz  re_check       ; al<>0 이면 반복  75H=01110101b  FCh(11111100b)→ -4 즉 IP=0006-4 = 2로 점프
22 0006 90                  nop
23
24                      ;-------  프로그램 종료 -------------------
25 0007                 exit_program:
26 0007 B4 4C               mov ah, 4ch         ; 즉치(표현 방법이 다름) ah 레지스터에 4Ch 설정.  B4h = 10110100b
27 0009 CD 21               int 21h             ; 프로그램 종료용 MS-DOS 시스템 콜
28
29 000B                 main endp
30 000B                 cseg ends
31                          end main
```

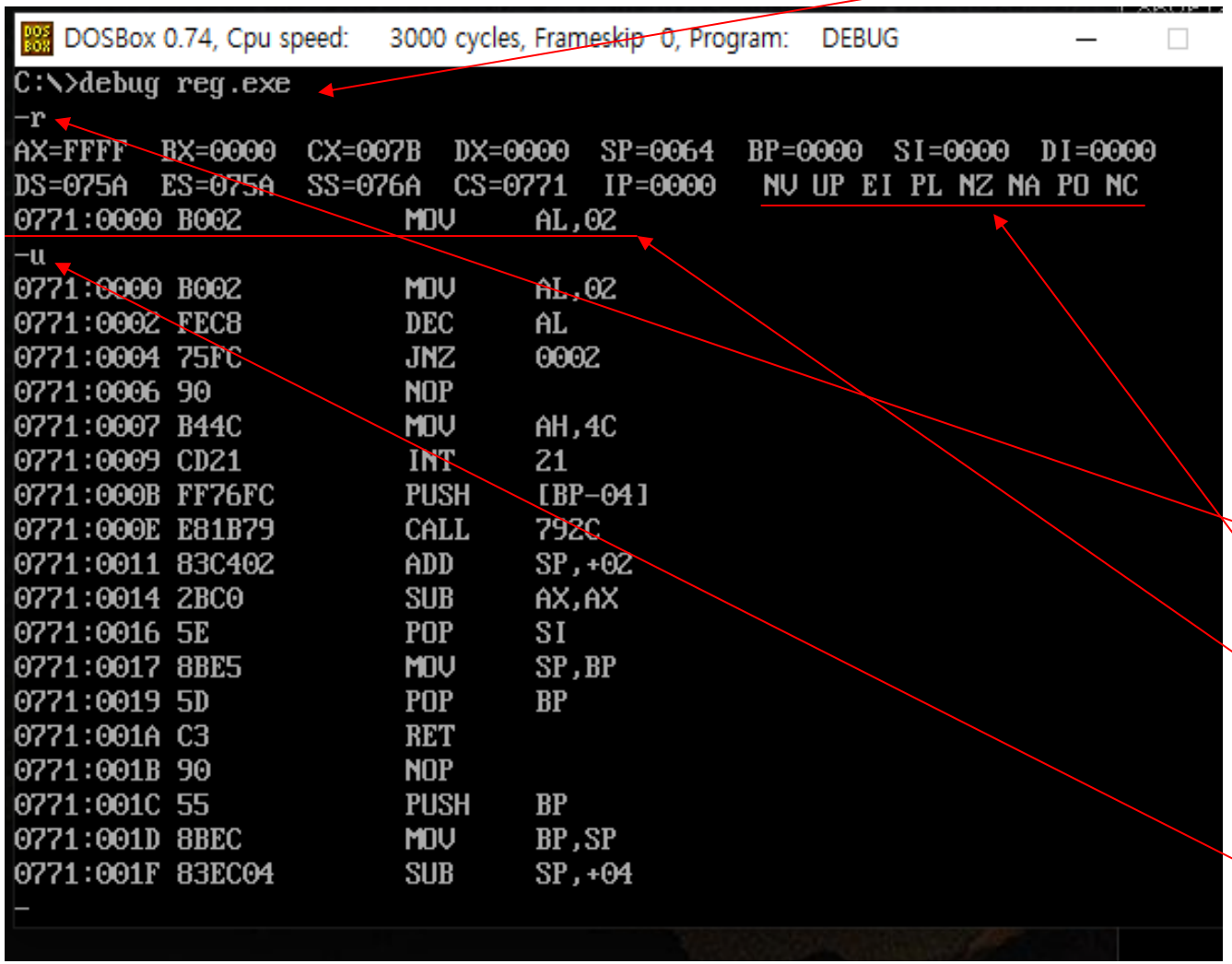
6비트 연산자(Op-code)

상대 주소지정 방식

상대 주소지정 모드(16비트) 파이프라인 이라면?

4. reg.asm 프로그램

- **debug 프로그램**



DOSBox 화면에서 "DEBUG reg.exe"를 입력

< DEBUG 명령어 >

"**u**" : 기계어 리스트 확인

"**r**" : 현재의 레지스터들을 표시

"**p**" : 프로그램을 한 줄씩 실행

"**d**" : 메모리 내용 디스플레이

"**q**" : 디버거 프로그램의 종료

현재의 레지스트 값 확인 → r 명령

상태레지스터의 현재 값 표시

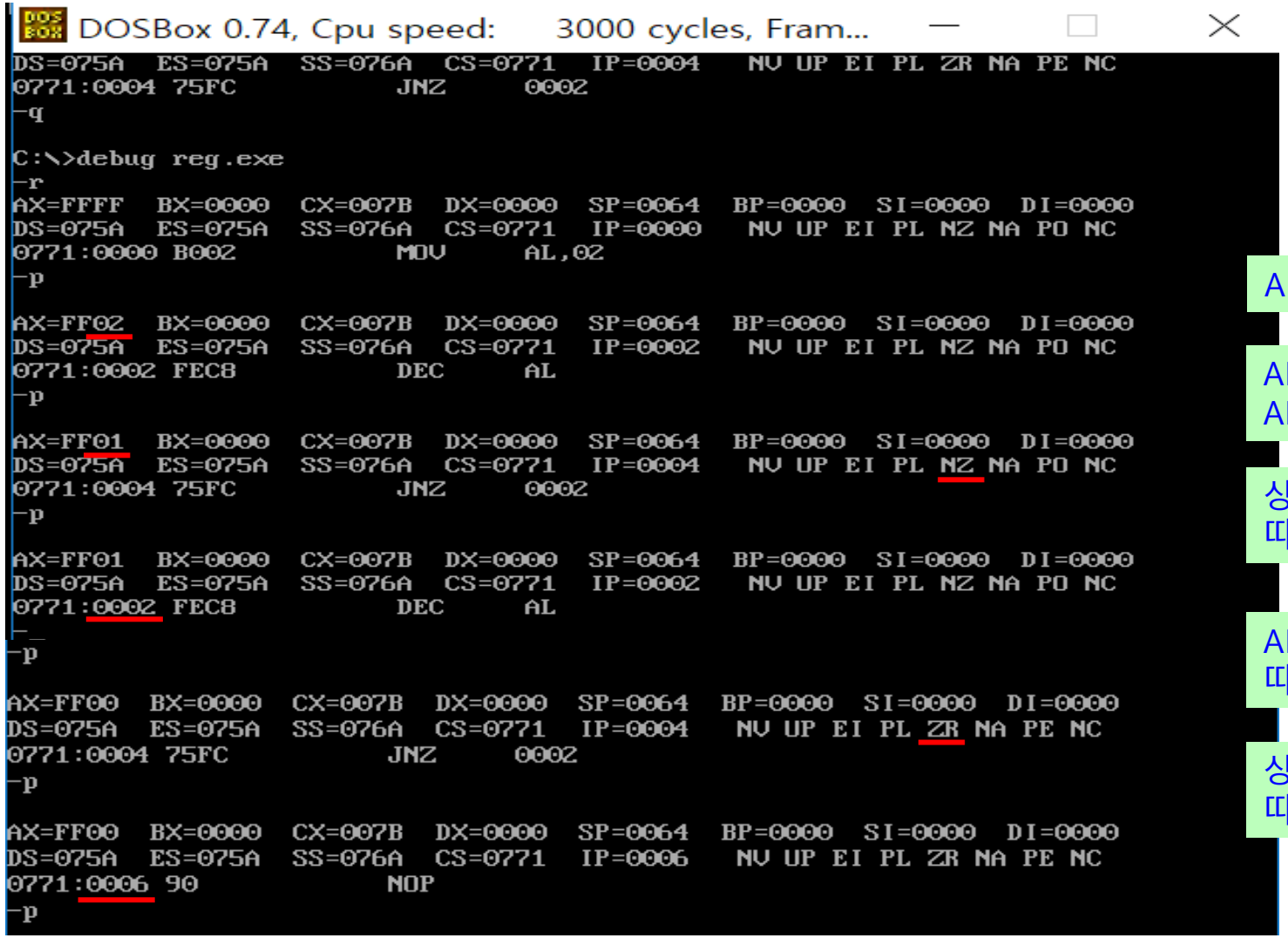
현재 실행될 명령어의 표시

실행될 명령어(기계어) 리스트
→ u 명령

4. reg.asm 프로그램

- **debug 프로그램을 통한 실행분석**

DOSBox 화면에서 "DEBUG reg.exe"를 입력
p 명령을 입력(한줄 실행)하면서 레지스터의 변화들을 확인



AL = 2

AL = AL - 1 = 2 - 1 = 1
AL =1 이므로 NZ(Not Zero)

상태 레지스터 NZ(Not Zero)
따라서 실행 후 2 번지로 점프

AL = AL - 1 = 1 - 1 = 0
따라서 실행 후 ZR로 변경

상태 레지스터 ZR(Zero)이므로
따라서 실행 후 점프하지 않음

5. STACK 프로그램

■ STACK 프로그램의 실행파일 생성 및 DEBUG 실행

- ✓ 윈도우에서 D:₩8086를 연다
- ✓ 어셈블러 실행
  - ➢ DOS 창에서 "MASM stack" 을 입력. OBJ는 자동으로 잡힘, 필요시 LST, CRF를 물을 때 stack을 입력
  - ➢ 또는 한번에 "MASM stack,,,," 를 입력
  - ➢ 윈도우에서 LST 파일의 내용을 확인(메모장 이용) → 여기에서 소스와 기계어를 모두 확인할 수 있다
    - ❖ 연산코드, 오퍼랜드, 조건부 이동 등등....
- ✓ 실행파일의 생성
  - ➢ DOS 창에 "LINK stack"을 입력하고 무조건 엔터키를 계속 입력 → stack.exe의 생성
- ✓ 디버거를 이용한 파일의 실행
  - ➢ DOS 창에서 "debug stack.exe"를 입력하고 엔터키를 쳐서 디버거 프로그램을 실행

5. STACK 프로그램

- STACK.asm

```
;----------------------
CR          equ 0dh
LF          equ 0ah
;----------------------
sseg segment para stack 'stack'
    db 64h dup(?)
sseg ends
;----------------------
dseg segment
data1 db 'Stack Test',CR,LF,'$'        ; 화면에 출력할 스트링
dseg ends
;----------------------
cseg segment
main proc far
    assume ss:sseg, ds:dseg, cs:cseg

    mov ax, dseg
    mov ds, ax              ;dseg 설정

    mov ax, 1234h           ; ax 레지스터에 1234h 값 넣음
    mov dx, 5678h           ; dx 레지스터에 5678h 값 넣음

    push ax                 ; ax 내용을 스택에 저장
    push dx                 ; dx 내용을 스택에 저장

;------  DS:SI의 내용을 화면에 연속출력('$' 만날때까지) ---
    cld                     ; 인덱스 레지스터 자동증가 설정
    mov si, offset data1
```

```
display_loop:
    lodsb                   ; al <- ds:si
                            ; & si=si+1(자동증가 어드레싱 모드)
    cmp al,'$'
    je    display_end       ; if $ then exit loop

    mov dl,al
    mov ah,02h
    int   21h               ; DL(ascii) 값을 화면에 표시
    jmp display_loop

display_end:
    pop   dx                ; 스택에 저장된 dx를 복원
    pop   ax                ; 스택에 저정된 ax를 복원

; ----- pop 명령은 먼저 push 된 순서로 먼저 꺼내진다
    push ax                 ; ax = 1234h
    push dx                 ; dx = 5678h
    mov  ax,0
    mov  dx,0
    pop  ax                 ; 나중에 저장된 5678h가 ax로 복원
    pop  dx                 ; 그 이전에 저장된 1234h가 dx로

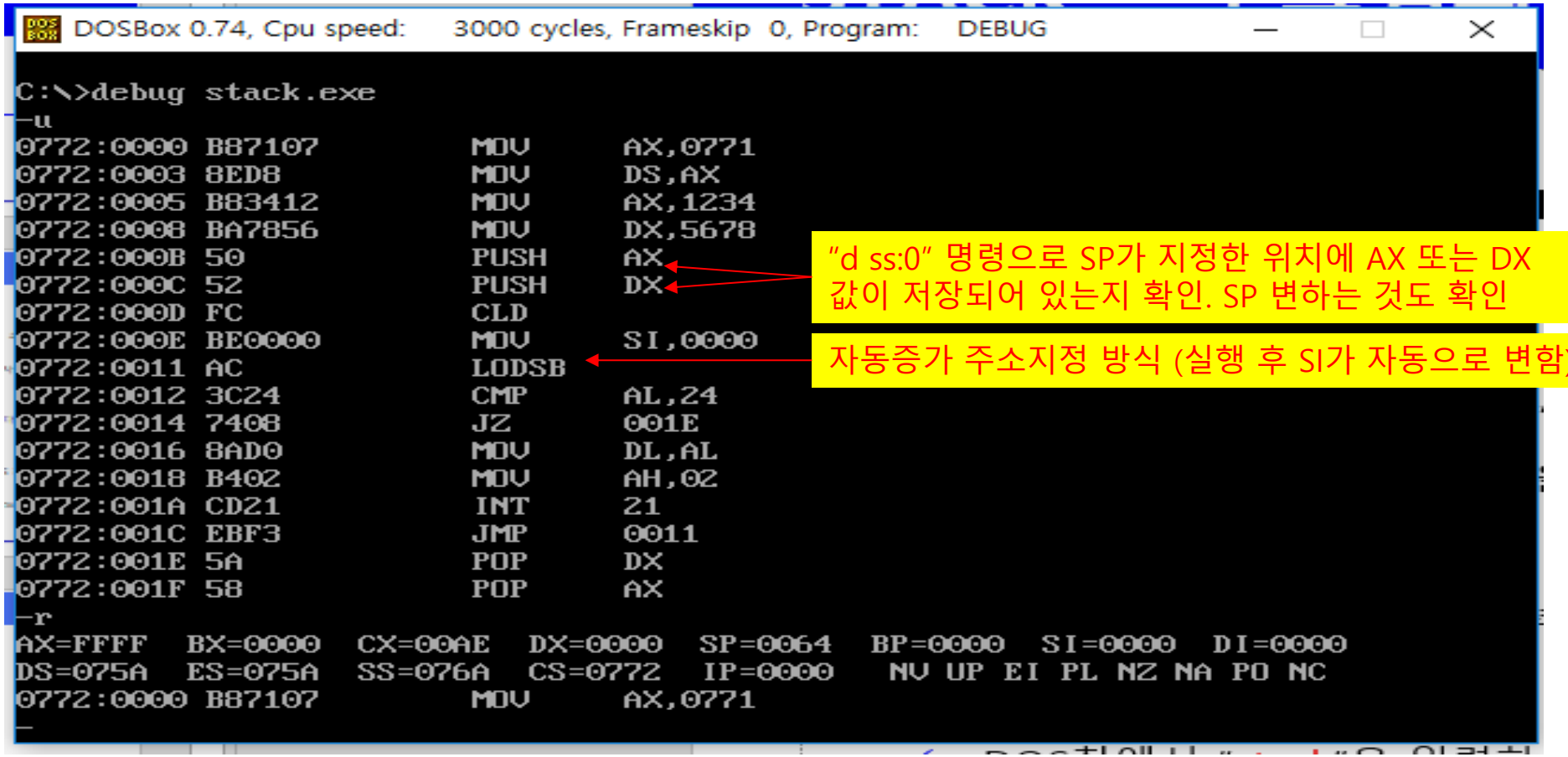
;-------  프로그램 종료 -------------------
    mov ah, 4ch
    int 21h                             ;프로그램 종료

main endp
cseg ends
    end main
```

5. STACK 프로그램

- DEBUG를 이용한 stack.exe실행
  - ✓ 각종 레지스터들의 현재 값을 볼 수 있다
  - ✓ P 명령으로 한 줄씩 실행하면서 레지스터 변화 관찰 → 끝까지 실행 후 q 명령을 종료

P 명령으로 명령어를 한줄씩 실행시키면서 레지스터의 변화 확인.
특히 PUSH, POP 명령어 실행 전후의 SP 값 확인



"d ss:0" 명령으로 SP가 지정한 위치에 AX 또는 DX 값이 저장되어 있는지 확인. SP 변하는 것도 확인

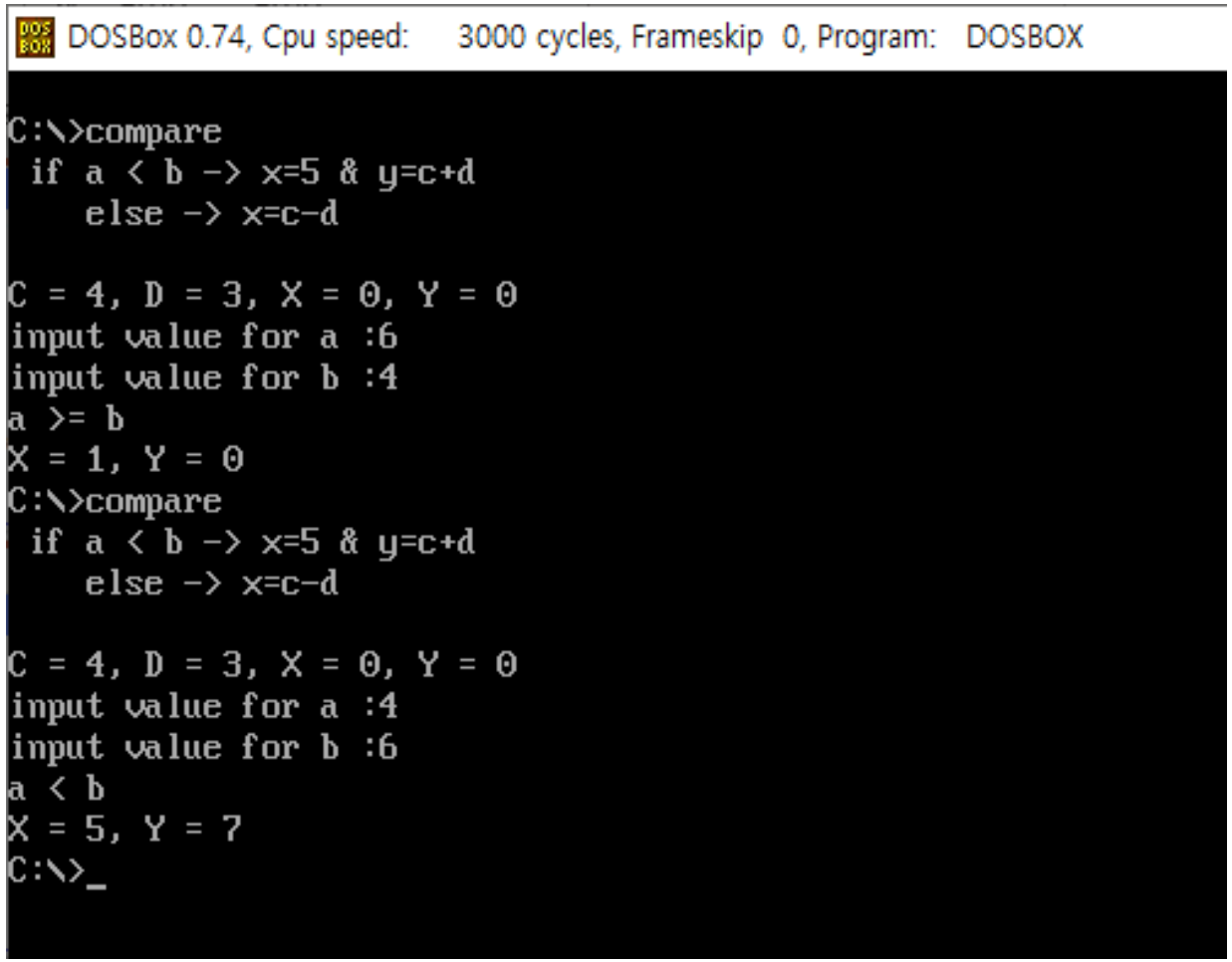
자동증가 주소지정 방식 (실행 후 SI가 자동으로 변함)

6. COMPARE 프로그램

- 다음 c 프로그램을 8086 어셈블리 언어로 작성한 소스

```
if (a<b) {
        x = 5;
        y = c+d;
}
else x = c-d;
```

- 어셈블러 실행 → "MASM compare,,,,"
- 링커의 실행 → "LINK compare"
- 프로그램의 실행 → "compare"
- 윈도우에서 메모장으로 compare.asm 을 분석
  - ✓ 필요시 진행



6. COMPARE 프로그램

다음의 수식을 8086 어셈블리어를 작성한 경우

```
if (a<b) {
   x = 5;
   y = c+d;
}
else x = c-d;
```

```
;---------------- compare.asm  ----------------------------------------------------------------
CR              equ             0dh
LF              equ             0ah

;-----------------------
sseg segment para stack 'stack'
    db 64h dup(?)
sseg ends

;-----------------------
dseg segment

value_a         db 2
value_b         db 1
value_c         db 4
value_d         db 3

value_x         db 0
value_y         db 0

msg_start       db ' if a < b -> x=5 & y=c+d',CR,LF
                db '     else -> x=c-d',CR,LF,'$'

msg_c           db CR,LF, 'C = $'
msg_d           db ', D = $'
msg_x           db ', X = $'
msg_y           db ', Y = $'

msg_x_out       db CR,LF,'X = $'

msg_a_in        db CR,LF,'input value for a :$'
msg_b_in        db CR,LF,'input value for b :$'


msg_ge          db CR,LF,'a >= b$'
msg_les         db CR,LF,'a < b$'

dseg ends
```

```
;-----------------------
cseg segment
main proc far
    assume ss:sseg, ds:dseg, cs:cseg

    mov ax, dseg
    mov ds, ax                          ;set Data Segment

;------------  display message "Condition " ------------
    mov ah,09                           ;ah=09: disp string from dx
    mov dx, offset msg_start
    int 21h                             ;DOS Function Call 실행

;------------  display message "C = " ------------
    mov ah,09                           ;ah=09: disp string from dx
    mov dx, offset msg_c
    int 21h                             ;DOS Function Call 실행
;---- display value 'c'  ----------------------------
    mov dl,value_c
    add  dl,'0'
    mov ah,02h
    int   21h           ;Dispay DL
;------------  display message "D = " ------------
    mov ah,09                           ;ah=09: disp string from dx
    mov dx, offset msg_d
    int 21h                             ;DOS Function Call 실행
;---- display value 'd'  ----------------------------
    mov dl,value_d
    add  dl,'0'
    mov ah,02h
    int   21h           ;Dispay DL
;------------  display message "X = " ------------
    mov ah,09                           ;ah=09: disp string from dx
    mov dx, offset msg_x
    int 21h                             ;DOS Function Call 실행
;---- display value 'x'  ----------------------------
    mov dl,value_x
    add  dl,'0'
    mov ah,02h
    int   21h                           ;Dispay DL
;------------  display message "Y = " ------------
    mov ah,09                           ;ah=09: disp string from dx
    mov dx, offset msg_y
    int 21h                             ;DOS Function Call 실행
```

6. COMPARE 프로그램

```
;---- display value 'y'  ---------------------------
    mov dl,value_y
    add  dl,'0'
    mov ah,02h
    int   21h                       ;Dispay DL

;------------  display message "Input a" -----------
    mov ah,09                       ;ah=09: disp string from dx
    mov dx, offset msg_a_in
    int 21h                         ;DOS Function Call 실행
;--------  get value a from KBD ------------------
    mov ah,01
    int 21h                         ;get calue(al) from KBD
    sub al,'0'                      ;convert ASCii -> value
    mov ds:value_a,al               ;save value a
;------------  display message "Input b" -----------
    mov ah,09                       ;ah=09: disp string from dx
    mov dx, offset msg_b_in
    int 21h                         ;DOS Function Call 실행
;--------  get value b from KBD ------------------
    mov ah,01
    int 21h                         ;get calue(al) from KBD
    sub al,'0'                      ;convert ASCii -> value
    mov ds:value_b,al               ;save value a

    mov al, ds:value_a              ; al <- a 값
    cmp al, ds:value_b              ; a값 >= b값
    jae case_a_large_equal          ; 크거나 같으면 점프

 ;-----   a 값이 b 보다 작은 경우 ----------------

    mov value_x,5
    mov al, value_c
    add al, value_d
    mov value_y, al                 ; y <- c + d
    mov dx, offset msg_les
    jmp end_point

;-----   a 값이 b 보다 크거나 같은 경우 ----------------
case_a_large_equal:
    mov al, value_c
    sub al, value_d
    mov value_x, al                 ; x <- c - d
    mov dx, offset msg_ge

end_point:
    mov ah,09                   ;ah=09: disp string from dx
    int 21h                                         ;DOS Function Call 실행

;------------  display message "X = " ------------
    mov ah,09                                       ;ah=09: disp string from dx
    mov dx, offset msg_x_out
    int 21h                                         ;DOS Function Call 실행
;---- display value 'x'  ---------------------------
    mov dl,value_x
    add  dl,'0'
    mov ah,02h
    int   21h           ;Dispay DL
;------------  display message "Y = " ------------
    mov ah,09                                       ;ah=09: disp string from dx
    mov dx, offset msg_y
    int 21h                                         ;DOS Function Call 실행
;---- display value 'y'  ---------------------------
    mov dl,value_y
    add  dl,'0'
    mov ah,02h
    int   21h           ;Dispay DL

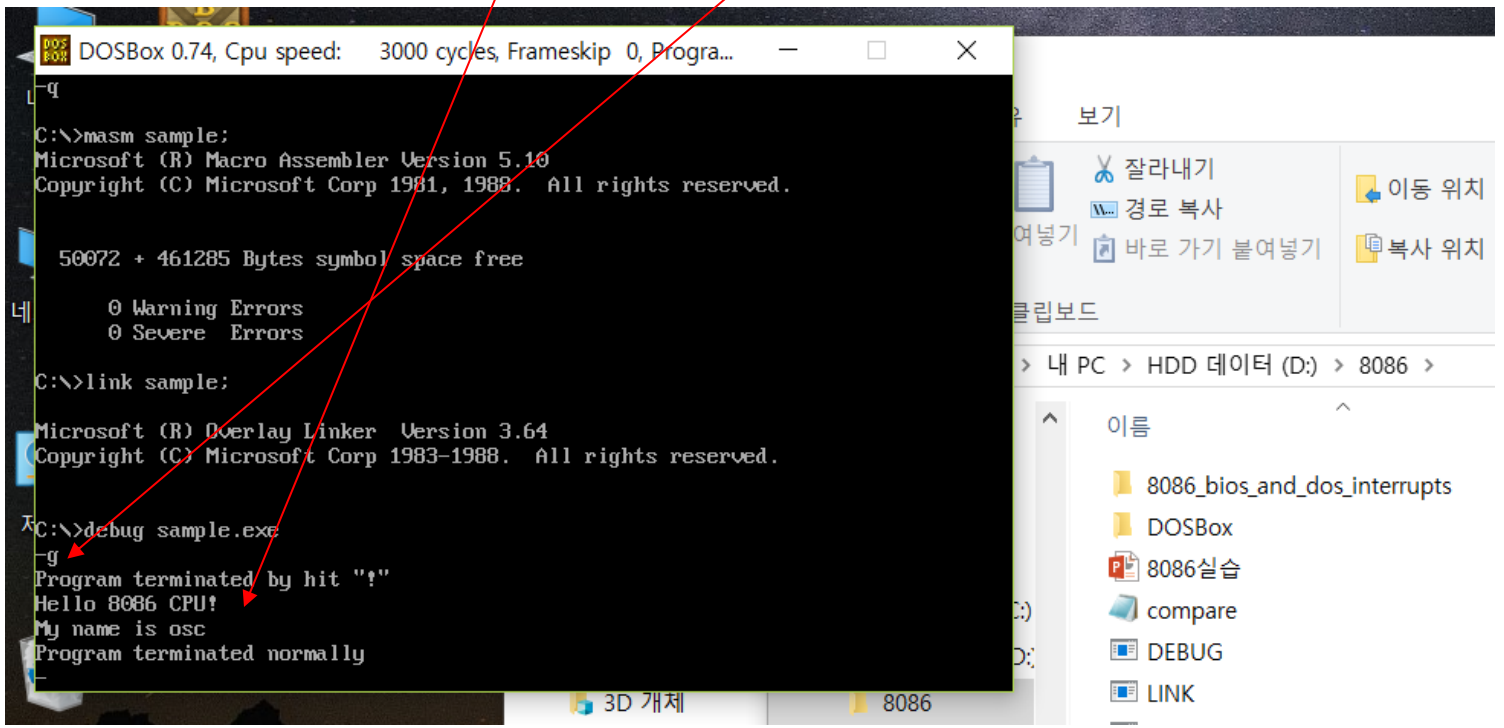
;-----------  Program End ------------------------
    mov ah, 4ch
    int 21h                                         ;프로그램 종료

main endp
cseg ends
    end main
```

7. 이름 출력 프로그램

- SAMPLE.ASM 프로그램

1) sample.asm 을 열고 프로그램을 확인한 후
2) 어셈블러와 링커를 이용해 실행파일(sample.exe)을 만듦
3) DEBUG 프로그램으로 sample.exe 프로그램을 로드 후 g 명령으로 실행
4) 여러분의 이름 등을 영어로 입력후 느낌표를 입력하면 프로그램이 종료됨



7. 이름 출력 프로그램

■ SAMPLE.ASM 프로그램

```
CR              equ 0dh
LF              equ 0ah

sseg segment para stack 'stack'
    db 64h dup(?)
sseg ends

;----------------------
dseg segment
data1 db 'Program terminated by hit "!"',CR,LF
        db 'Hello 8086 CPU!',CR,LF
        db 'My name is ', '$'
dseg ends

;----------------------
cseg segment
main proc far
    assume ss:sseg, ds:dseg, cs:cseg

    mov ax, dseg
    mov ds, ax

;------  DS:DX의 내용을 화면에 연속출력('$' 만날때까지) ---
    mov ah, 09
    mov dx, offset data1
    int 21h                     ;DOS Function Call 실행

;------  Get 1 character from KBD w/o echo ----------
read_loop:
     mov ah,07h
     int   21h                  ;Read 1 char from KBD (AL)

;------ if input = '!' then exit program ------------
     cmp al,'!'
     je  exit_program

;------ Display character ----------------------
     mov dl,al
     mov ah,02h
     int   21h                  ;Display DL
     jmp read_loop

;-------  프로그램 종료 -------------------
exit_program:
    mov ah, 4ch
    int 21h                     ;프로그램 종료

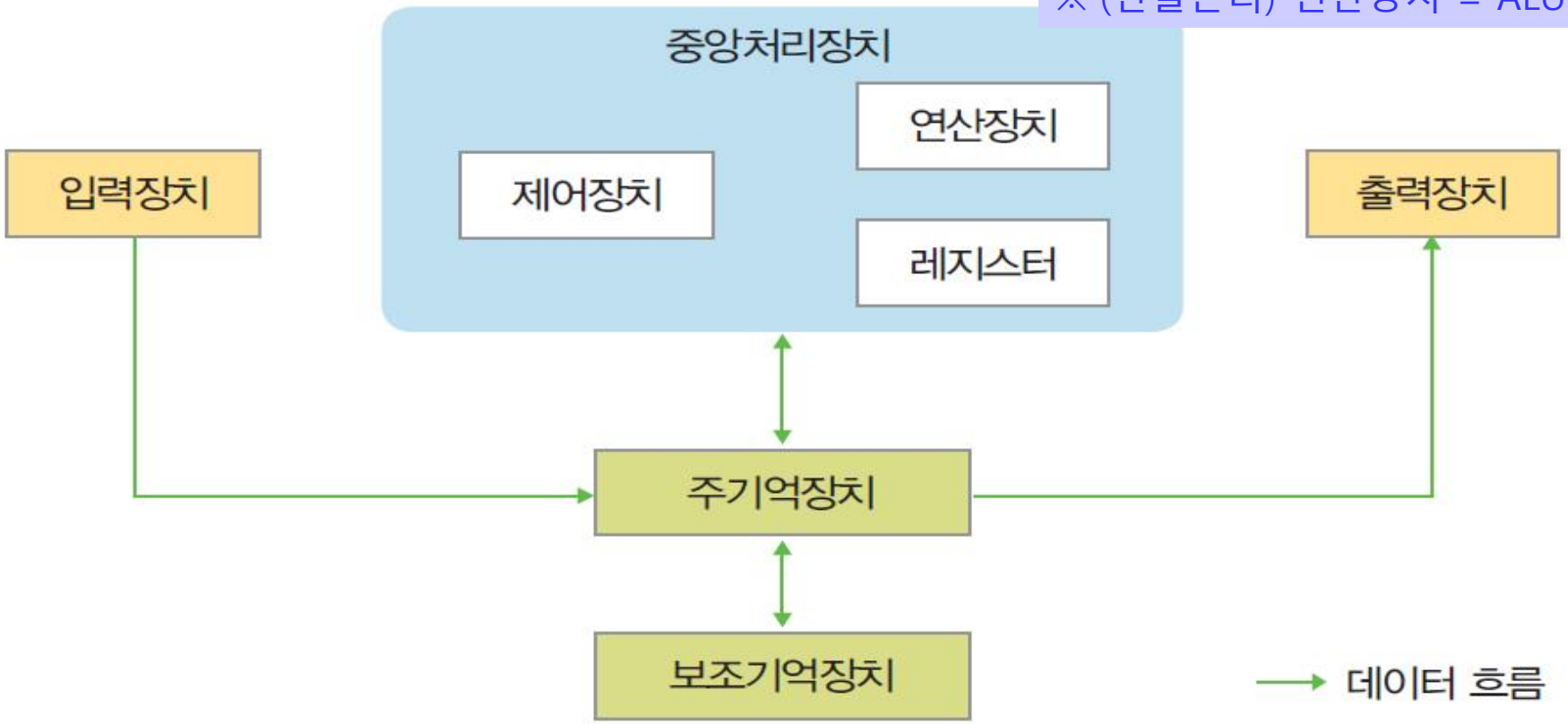
main endp
cseg ends
    end main
```

Thank you

참고. 일반 CPU 기본 사항

- **컴퓨터 하드웨어의 구성**

※ (산술논리) 연산장치 = ALU



하드웨어의 구성

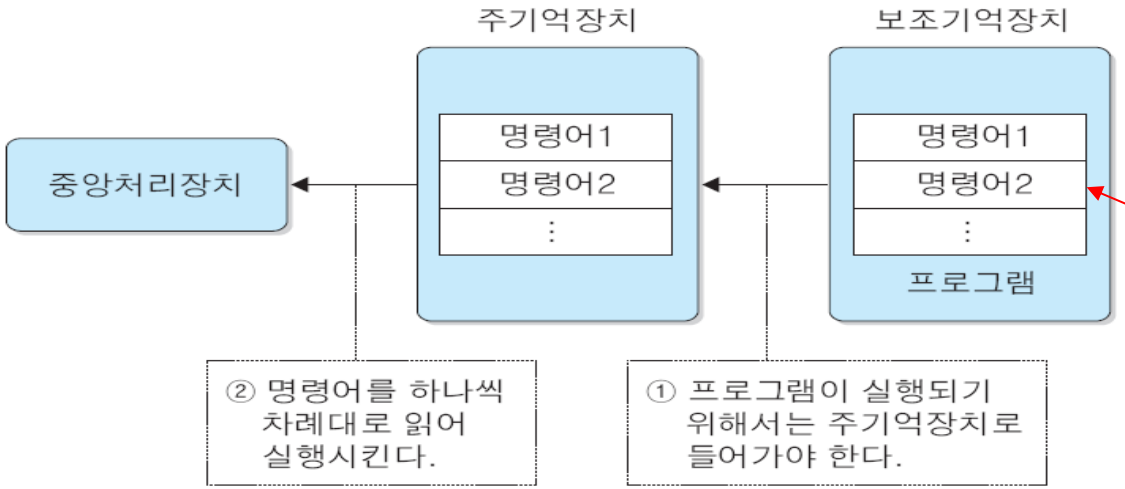
참고. 일반 CPU 기본 사항

■ 컴퓨터 프로그램의 실행

✓ 부팅(Booting) 절차



✓ 프로그램의 실행 절차



High-level 언어

High-level language program (in C)

```
swap(int v[], int k)
{int temp;
   temp = v[k];
   v[k] = v[k+1];
   v[k+1] = temp;
}
```

Compiler

Assembly 언어

Assembly language program (for MIPS)

```
swap:
        muli $2, $5,4
        add  $2, $4,$2
        lw   $15, 0($2)
        lw   $16, 4($2)
        sw   $16, 0($2)
        sw   $15, 4($2)
        jr   $31
```

Assembler

기계어(이진수)

Binary machine language program (for MIPS)

```
00000000101000010000000000011000
00000000000110000001100000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
00000011111000000000000000001000
```

참고. 일반 CPU 기본 사항

■ CPU의 구성 (중앙처리장치, Central Processing Unit)

최근의 CPU 구조에서는 하나의 CPU 칩 안에 여러 개의 CPU가 포함되기 때문에 기존의 CPU의 구성 내용인 제어장치, ALU, 레지스터들을 묶어 **코어(core)** 라고 부른다

✓ CPU는 다음과 같은 **3가지의 주요 기능을 갖는 논리회로의 블록**으로 구성

1) 제어장치 (Control Unit)

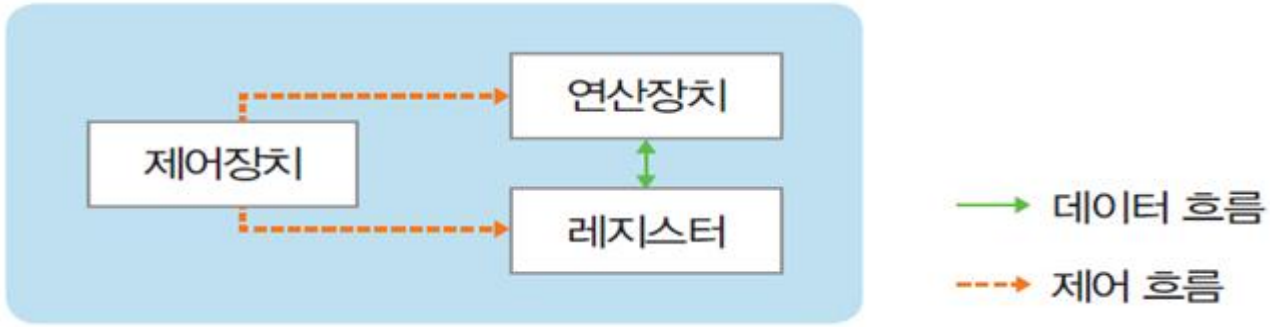
❖ 명령어를 순서대로 실행할 수 있도록 제어하는 장치 → **CPU 내부의 CPU**

2) 연산장치 (ALU=Arithmetic & Logic Unit)

❖ 덧셈/뺄셈/곱셈/나눗셈 등의 **산술연산**, 논리 곱/합/부정 등의 **논리연산**만을 전문적으로 수행

3) 레지스터 (Register)

❖ **CPU 내부의 초고속의 기억장치**로 명령어 주소, 명령어 코드, 연산에 필요한 데이터, 연산 결과 등을 임시로 저장하는 매우 빠른 기억장치



중앙처리장치의 구성

참고. 일반 CPU 기본 사항

■ CPU 동작의 기본 원리

✓ CPU는 현재 수행 중인 프로그램이 어떠한 내용인지를 전혀 모르며 단지 컴파일러/어셈블러에 의해 만들어진 실행 파일 즉, 기계어(명령어)의 집합을 하나씩 읽어 오고 실행한다.

- ➢ 운영체제 인지 사용자 응용 프로그램인지만 구분

1) 인출 단계

- ➢ CPU는 PC(프로그램 카운터)가 가리키는 기억장치에서 프로그램 코드(이진수 명령어) 하나를 읽어 온다
- ➢ 읽어온 명령어를 내부 IR(명령어 레지스터)에 저장하고 PC 값을 자동으로 증가시킴

2) 실행 단계

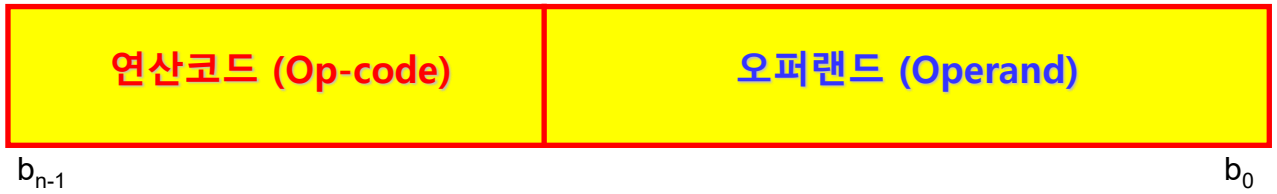
- ➢ CPU는 IR 레지스터의 내용에 있는 기계어(명령어)를 해독해 해당 명령어를 실행
- ➢ 내부의 레지스터들을 이용해 명령어를 실행하며 부득이한 경우 외부 기억장치를 임시 데이터 저장장소로 사용
- ➢ CPU가 임시로 레지스터의 값을 보관할 때는 주기억장치의 스택 영역을 이용하며 스택 포인터(SP) 레지스터로 관리
  - ❖ push, pop 명령을 사용하며 이때 SP 값이 자동으로 증가/감소 된다.
- ➢ 명령어 실행시의 연산 결과는 CPU 내부 상태 레지스터에 기록하여 뒤따르는 명령어에서 필요시 사용하게 함

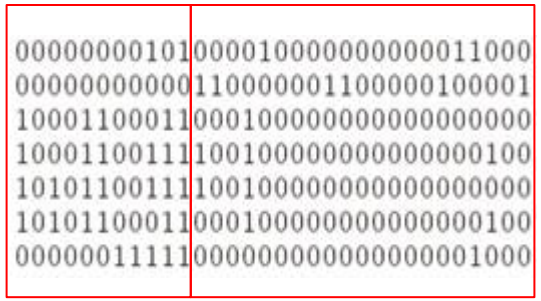
참고. 일반 CPU 기본 사항

■ 명령어(기계어)의 형식

✓ 2진수인 명령어(기계어)는 연산코드(연산자) 필드와 오퍼랜드 필드로 구성

✓ CPU 설계시에 결정되며 CPU 마다 연산코드/오퍼랜드 필드의 크기가 다름





✓ 연산 코드(Op-code: Operation code)

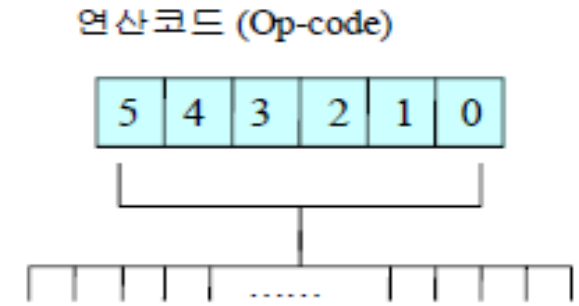
- CPU가 수행할 연산(명령)의 종류를 지정해 주는 비트들
- 비트 수 = '3' 이면, 지정될 수 있는 연산의 최대 수: 2^3 = 8개

✓ 오퍼랜드(Operand)

- 연산 코드가 실행되는데 필요한 숫자, 주소 등의 정보를 포함하는 필드
- 연산코드에 따라 오퍼랜드의 종류가 결정된다
- 비트 수 = '5' 라면, 주소지정(addressing) 할 수 있는 기억 장소의 최대 수: 2^5 = 32 개

참고. 일반 CPU 기본 사항

■ 명령어(기계어)의 형식 - 연산코드(op-code)



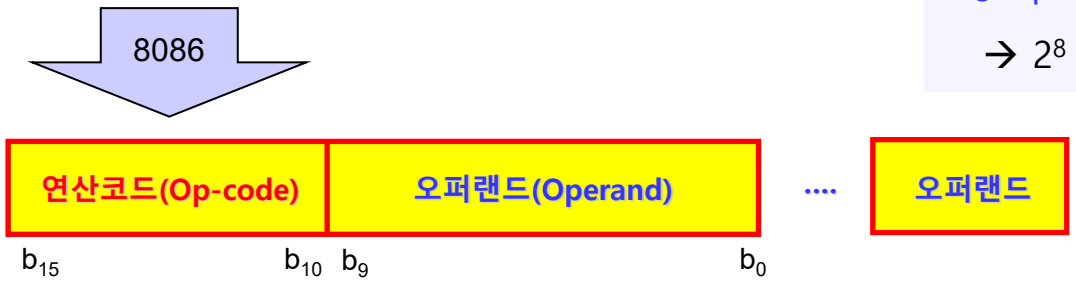
8086 기계어

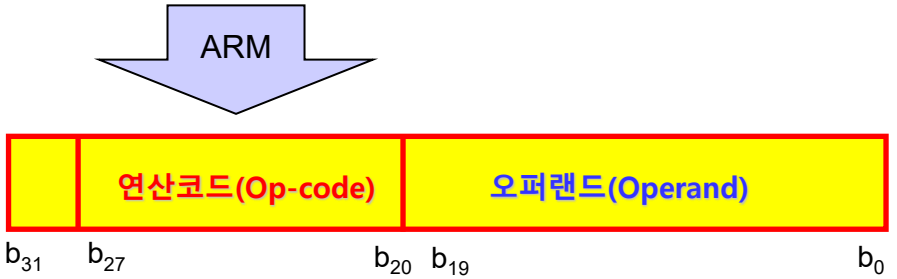
인텔 8086 CPU: 가변길이 명령어 (CISC의 특징)

- 6 비트의 연산코드를 가진다
  → 2^6 = 64 가지의 연산을 정의 가능
- 명령어 길이는 1~6(8~48비트) Byte로 명령어마다 다름
  → 연산코드에 따라 오퍼랜드의 길이가 달라짐 → 다양한 문제 발생

ARM 프로세서: 고정길이 명령어 (RISC의 특징)

- 명령어의 길이는 항상 32비트(4 Byte=1워드)
- 8 비트의 연산코드
  → 2^8 = 256 가지의 연산을 정의 가능





참고. 일반 CPU 기본 사항

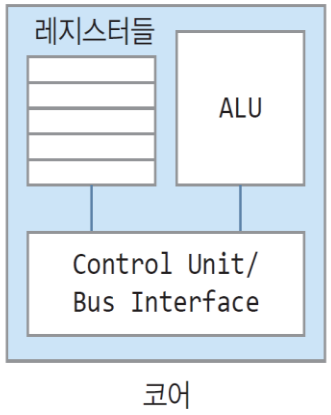
돌아가기

■ 멀티 코어 CPU(Multi-core CPU)

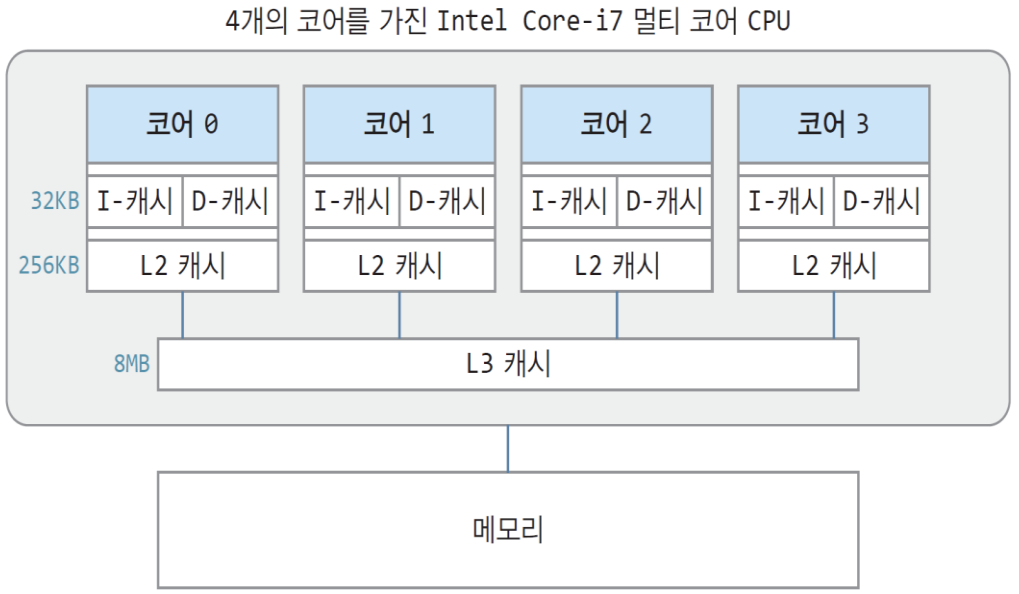
✓ 2001년 IBM에 의해 PowerPC라는 멀티코어 CPU 개발

- CPU 내부에 2개의 프로세서(processor) 포함
- 2개의 프로그램을 동시에 실행(병렬성)
- 코어는 완벽한 처리기(과거 개념의 CPU)

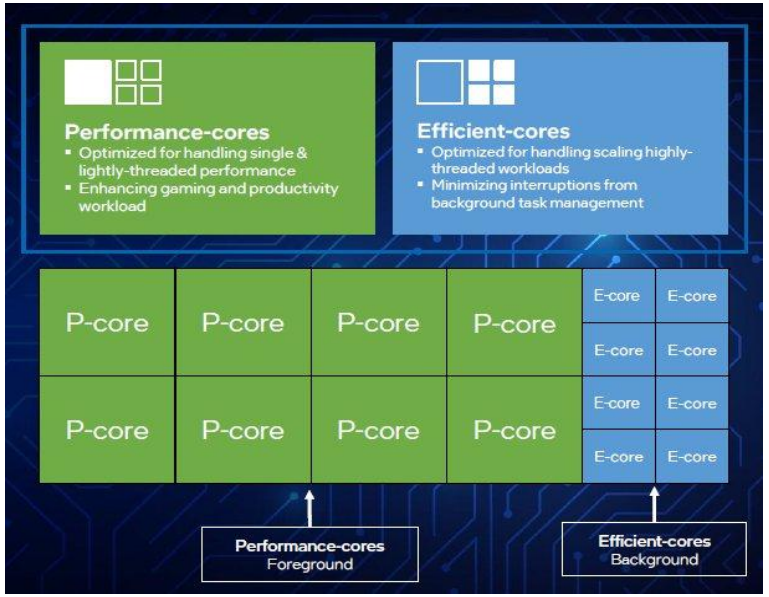
최근의 CPU 구조에서는 하나의 CPU 칩 안에 여러 개의 CPU가 포함되기 때문에 기존의 CPU의 구성 내용인 제어장치, ALU, 레지스터들을 묶어 **코어(core)** 라고 부른다



(a) 코어의 내부 구조



(b) 4개의 코어를 가진 Intel Core-i7 멀티 코어 CPU 사례



< 인텔 12세대 이후 CPU: P Core, E core >